# Warehouse Robot Task Allocation using Dueling Double Deep Reinforcement Learning

Himnish Sai Kommana
*School of AI*
*Amrita Viswa Vidyapeetham*
Bengaluru, India
bl.en.u4aid23018@bl.students.amrita.edu

T Sai Swaroop
*School of AI*
*Amrita Viswa Vidyapeetham*
Bengaluru, India
bl.en.u4aid23051@bl.students.amrita.edu

V Dasa Manoj
*School of AI*
*Amrita Viswa Vidyapeetham*
Bengaluru, *India*
bl.en.u4aid23055@bl.students.amrita.edu

***Abstract*—The needs of the modern automated warehousing system demand that intelligent systems can be employed to accomplish their objectives because they need to function by interacting with a number of autonomous mobile robots operating in a dynamic environment. Due to the nature of the environment in which the operations occur, heuristics would prove to be inefficient for routing because they are not capable of incorporating changing orders as well as energy needs of the system while considering its maximum capacity. In the study, we introduce a novel system called TA-RWARE that is based on multi-agent reinforcement learning to handle task allocation and autonomy of the mobile robots using its DDQN policy.**

**In our architecture, there are several agents running in a modular grid-based warehouse that includes agents for operating AGVs and picker agents. Battery-aware routing together with dynamic order injections, reward schemes, and automatic picking and dropping procedures have been used in order to overcome the problem of learning from sparse rewards.**

**In order to assess the efficacy of the proposed DDQN policy, a series of experiments were performed in which the policy was compared against other baselines such as greedy and random policies using deterministic seeds.The project additionally integrates a Streamlit-based visualization.**

***Index Terms*—Reinforcement Learning, Deep Q-Network, Warehouse Automation, Multi-Agent Reinforcement Learning, AGV Routing, Task Allocation**

## I. Introduction

Modern times have witnessed a drastic increase in developments related to e-commerce and logistics. Hence, the automatic process of managing the warehouse becomes one of the focal points of research in the field of intelligent logistics. The modern warehouse requires robots and robotic agents that can operate in an unpredictable environment considering such factors as routing, task allocation, congestion, recharging, and order completion. On the contrary, traditional routing solutions usually involve heuristic routing without the possibility of adaptation to dynamic occurrences, such as battery depletion, order insertion, traffic congestion, and others.

Nevertheless, a new method of addressing similar challenges may be provided as discussed in this paper, which involves the utilization of reinforcement learning approaches rather than traditional approaches described above. As opposed to supervised learning, reinforcement learning does not imply the presence of training datasets that are labeled as positive, negative, good, bad, or anything similar. Instead, it implies rewarding and punishing the agents for completing or not completing particular actions.

The current project will be referred to as TA-RWARE (Task Allocation Reinforcement Learning for Warehouse Autonomous Routing and Execution). This project uses an existing benchmark framework called RWARE and provides innovative approaches in multi-agent reinforcement learning for warehouses. Many innovative features that are rarely found in analogous projects are included in our approach. For example, these are battery-aware routing, heterogeneous AGV/picker collaboration, order insertion, cooperative reward system, and even switching from picking to delivering due to sparsity of rewards. In addition, a Dueling Double DQN model is used.

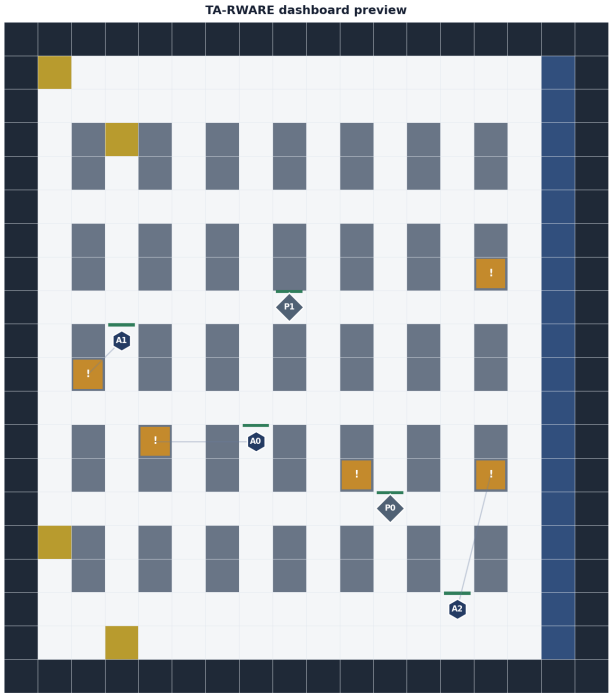


Fig. 1. Warehouse environment used showing AGVs, picker agents, racks, charging stations, and delivery zones.

## II. Problem Statement

It should be mentioned that a number of issues can arise when creating automated processes of warehousing operations. These include the impossibility of task allocation depending on the battery level, queues, traffic inside the warehouse, and other related issues. Thus, due to the issues mentioned above, there may be AGV idleness, delays in deliveries, and other related problems. The higher the traffic within the warehouse is, the greater the probability of agents intersecting while moving to the charging stations and delivery points.

For example, one issue concerns battery management and the application of a greedy algorithm, which results in short-term solutions at the cost of ignoring some energy-related issues in exchange for time reduction. The result is that a need to recharge simultaneously arises among many agents, causing traffic and deadlocks. In order to resolve this issue, it is necessary to consider the issue of using batteries dynamically, along with applying an instruction that will help control agents and guide them to charging stations.

Credit assignment should also be taken into account while developing a reinforcement learning mechanism. For instance, such an assigned task might consist of going to a particular rack where a package needs to be picked up, delivered, and moved to another rack for continuing the process. Only after all actions, there will appear some rewards for such an agent. This approach leads to further credit issues that need to be considered.

Thus, as a part of the project, it is supposed to configure an environment for AGVs including the use of batteries and order queues, train a Dueling DDQN model for achieving the coordination between AGVs and pickers, implement an automatic process of picking and dropping off packages, support order queue management, and utilize the Streamlit library in order to build a dashboard.

## III. MDP Formulation

The warehouse environment is modeled as a Markov Decision Process represented as:

$$MDP = (S, A, T, R, \gamma) \tag{1}$$

State space contains a 57 dimensional observation vector, which gets individually normalized for each agent. Details like the position of the agent, its energy level, whether it has any load or not, phase, direction, neighboring agents, orders, and a 5X5 perception map of the warehouse exist in the observation vector. State space includes the location of the agent, its energy level, manhattan distance from targets, locations of orders, neighbors, and local occupancy of the warehouse. It closely resembles the concept of Markov property since all environmental details necessary for making decisions are included in it.

Action space consists of five distinct actions:

$$A = \{North, South, East, West, Wait\} \tag{2}$$

TABLE I
Reward Structure and Rationale

| Reward Event | Value | Rationale |
|---|---|---|
| delivery_complete | +32.0 | Strong terminal signal — 4× the pickup reward to prioritise task completion |
| pickup_item | +8.0 | Milestone reward on reaching the rack |
| progress (move closer) | +0.6 | Dense guidance per Manhattan unit reduced toward target |
| move_away (move farther) | -1.2 | 2× progress weight — prevents oscillation near target |
| collision | -1.0 | Penalty for hitting walls, agents, or racks |
| battery_empty | -6.0 | Equivalent to ∼0.75 lost deliveries — strong deterrent |
| picker_assist | +2.5 | Picker agents earn this when within proximity of an active delivery goal |
| team_bonus | +55.0 | All agents receive this when every order in the episode is completed — cooperative signal |
| time_step | -0.012 | Small per-step penalty applied to all agents to discourage idle behaviour |

In contrast to normal warehouse setup, which requires manual pickup and drop of items, automated pickup and drop is done if the agent reaches the rack or target location. This simplifies things for the agent as well as facilitates smooth reward generation process.

Some examples of dynamics are Transition model that is deterministically specified using joint state and action selected. Some other dynamics include battery draining while moving, battery override, battery charging and dynamically generated tasks after every 120 simulated time-steps. Regarding battery draining dynamics, it is worth mentioning that the rate of battery draining while moving is 0.4 while remaining still it is 0.1. Any agent whose battery level drops below 20% is automatically routed to charging station.

The reward scheme consists of a combination of sparse and dense rewards for facilitating navigation and performance of tasks. For instance, 32 rewards are generated if delivery task is accomplished successfully while 8 rewards are generated after pickups. In addition, dense rewards are generated through both positive and negative rewards in order to facilitate efficient navigation. To be more precise, when the agent steps closer to the destination, a positive reward value of +0.6 is obtained. However, if it takes steps away from the target, it will be subject to a negative reward of 1.2, which is intended to prevent oscillations close to the goal point by being twice the reward value of reaching it. Each time step comes with a small negative reward value of 0.012, which acts as a punishment for all agents to avoid idling. Finally, there is also a team reward of +55.0 to all agents once all orders have been completed in each episode. The departure penalty is deliberately twice the value of the progress award to guarantee that the overall motivation will always encourage the direction towards the target. The group incentive of +55.0 is the key signal for cooperation; otherwise, individually trained robots would have no motivation to help one another because each robot's reward for delivery is adequate on its own.

Figure 2 illustrates the interaction cycle between warehouse agents and the environment.

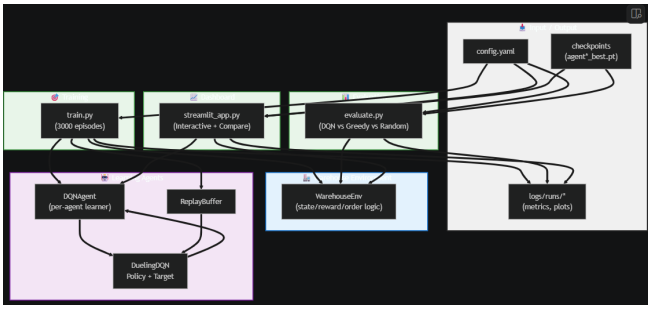


Fig. 2. Interaction pipeline between warehouse agents, replay memory, DDQN network, and environment.

## IV. Literature Survey

DRL has been demonstrated as an effective technique for dealing with sequential decision-making challenges in fields such as robotics, navigation, and autonomy. The key contribution of Mnih et al. included the creation of DQN (Deep Q-Network) in which Q-Learning algorithms and neural network models were combined to guarantee stability in training within complex environments. According to their study, neural networks can learn policies from environmental observations.

The contributions by Wang et al. led to the development of the Dueling DQN network structure where estimates of state values and action advantages were computed independently. This strategy helped enhance efficiency during interactions in situations where actions possessed the same utility levels. Furthermore, the double DQN proposed by Van Hasselt et al. was aimed at reducing overestimation of Q-values through decoupling of action selections and target estimations.

Christianson et al. developed RWARE (cooperative multi-agent warehouse reinforcement learning environment). However, current warehouse reinforcement learning benchmark architectures do not take into account practical issues such as battery recharging operations, agent heterogeneity, asymmetric reward systems, and interaction mechanisms. TA-RWARE (Task-Augmented RWARE) is an enhanced version of the current benchmark architectures incorporating battery recharge behaviors, task injection mechanisms, heterogeneous agents, and visualizations.

## V. System Architecture

The TA-RWARE framework consists of several modules including the warehouse environment, the observation encoder, the Dueling DDQN neural network, the experience replay memory, the policy evaluation, and the Streamlit dashboard. The warehouse environment involves using a tunable grid-world warehouse that consists of racks, delivery goals, charging stations, AGVs, and picking agents. The environment continuously tracks the position, energy level, collision, reward statistics, and assigned tasks.Picker agents act differently compared to AGVs. In contrast to AGVs, which are programmed with specific orders to reach racks and destinations, picker agents are not assigned any orders. They will get a bonus point of picker_assist (+2.5) upon reaching a delivery destination, just like humans do when working as assistants to operations performed at the docking station. Clearly, the difference in bonuses means that AGVs and pickers follow different policies in the same environment.

The observation encoder is used to transform the observations from the environment into normalized state inputs of the neural network. An agent senses its surroundings in the local/global warehouse by observing other agents in proximity, pending orders, and the orientation of the goal vector. The experience replay memory stores transition tuples in terms of state inputs, actions taken, rewards received, next states, and terminal status.

In addition to policy comparisons, the evaluation process involves comparing the policies against the greedy Manhattan distance heuristic and a random baseline. The Streamlit dashboard is used for the parallel evaluation of different policies and also shows some metrics including rewards, deliveries, energy level, and warehouse performance.

## VI. Deep Neural Network Architecture

The architecture of the proposed model includes a Dueling Double Deep Q-Network that uses independent learning methods for several agents. In the network structure, there is a common layer responsible for feature extraction in the state space, which is followed by two additional layers – the value layer and the advantage layer.

Q-value can be determined using the following formula:

$$Q(s,a) = V(s) + (A(s,a) - mean(A)) \tag{3}$$

The architecture of the neural network is comprised of two hidden layers that are interconnected through 256 and 128 nodes respectively, in addition to two channels for value stream and advantages stream. The neural network consists of Relu activation functions and Xavier initialization methods. DDQN method is applied to deal with the problem of overestimation of the Q-values and Huber loss function is adopted to mitigate the influence of high temporal difference values during training. Gradient clipping is implemented with a clip norm of 1.0 in case of an abrupt change in the reward function.

The individual instance of the DQNAgent is assigned to each agent of the warehouse without any weight sharing among different types of agents.

Figure 3 illustrates the Dueling Deep Q-Network architecture used.

## VII. Implementation Details

This environment was realized via using Python, NumPy, and PyTorch libraries. A warehouse is considered that consists of a 5*7 grid where there are three autonomous guided vehicles and two picking agents. Charging stations, racks, and delivery destinations are also located within the grid. In order to make energy consumption continuous, the movements of the agents would require 0.4 while resting would cost 0.1 energy units. In case the battery is less than 20%, the agent would have no other way but to reach a charging station.Besides using a single random seeding for global initialization, every time

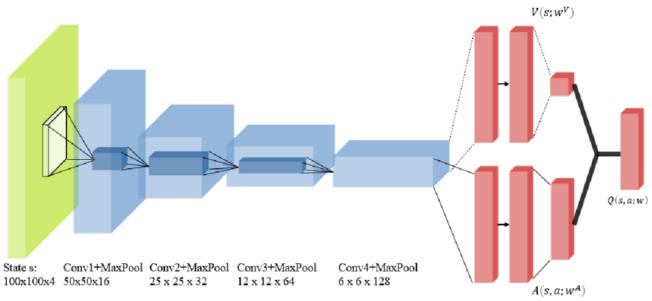


Fig. 3. Dueling Deep Q-Network architecture showing shared layers, value stream, and advantage stream.

we train the agent, the config_snapshot.yaml is created, which records the configuration with the exact hyperparameters used, such as reward weight, network structure, buffer size, and epsilon schedule. As a result, any training process can be exactly repeated by running train.py with the generated snapshot. When evaluating the agents, we always use a deterministic sequence of seeds, which are independent of the seed used for training.

The list of implementations comprises reset methods, dynamic tasks allocation, creating observations, collision detection, and intelligent agent behavior considering their batteries. Replay buffer is able to save 30,000 episodes that could be sampled. Mini-batches are generated from buffers, thus, making learning possible.

The reproducibility is secured by deterministic seeding for the Python, NumPy, and PyTorch environments. Configuration seeding and snapshotting were carried out for the benchmark reproducibilities. Also, unit tests were executed that include observation sizes, intelligent agent behavior regarding batteries, environment consistencies, and automatic picking abilities.

## VIII. Training and Hyperparameter Tuning

The agent undergoes training with 3000 episodes using a process called ε-greedy exploration. Here, the initial value of ε equals 1.0, but it decays to 0.05 over time. This approach guarantees that sufficient exploration occurs during the early stages of learning and adequate exploitation occurs towards the latter stages.

There were some parameters attempted in finding out the right combination of parameters required for effective learning. The parameters involved include learning rate of 0.0005, batch size of 64, buffer size of 30,000 and gamma of 0.97. From the experiments, it becomes clear that smaller hidden layers cannot be used to learn within an input space of 57 dimensions. However, using too large a neural network will make learning inefficient on CPU. In case of completion of all orders of the episode, all agents get a cooperative bonus worth +55.0. We arrived at this number by experimentation on 25, 55, and 80 numbers. 25 was too low compared to the individual reward for delivery and did not create a strong enough signal for cooperation. 80 made Q-values become unstable close to convergence because of the big TD error generated. The figure

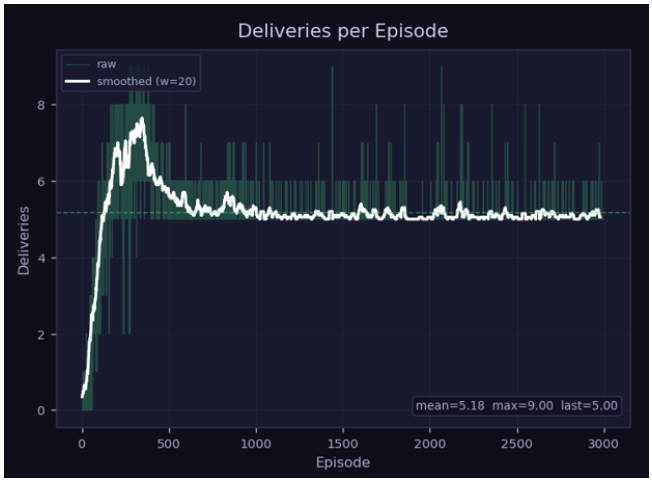


Fig. 4. Deliveries during 3000 episodes of training

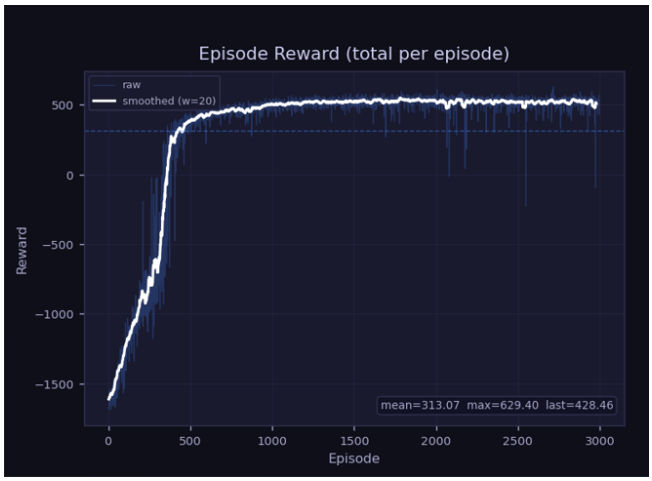


Fig. 5. Reward curve during the training phase

55 can be approximated by 1.7 times the individual delivery reward, which makes it good to motivate agents to help each other deliver orders.

Other reinforcement learning algorithms which have been experimented with during the process include PPO, QMIX, DQN without dueling network structure and independent Q-learning algorithms. The parallelization of PPO is too resource-intensive while centralized learning using QMIX can be quite complicated. Thus, Dueling DDQN algorithm is used because of its balance of learning and sample efficiency.

The system consists of three AGVs. Each of the 3 AGVs has a Dueling DQN agent.

Due to the epsilon values the agents try to explore more ways and causing the spike in the rewards and later settling down as they found the optimal way to reach the goal as depicted in Fig6 and Fig 7.

The number of steps for reaching the goal gradually decreased as the agent's find the optimal way i.e the less number of steps to reach the goal.

As the training proceeds the collision rate is has gradually


Fig. 6. Individual agent reward curve during the training phase

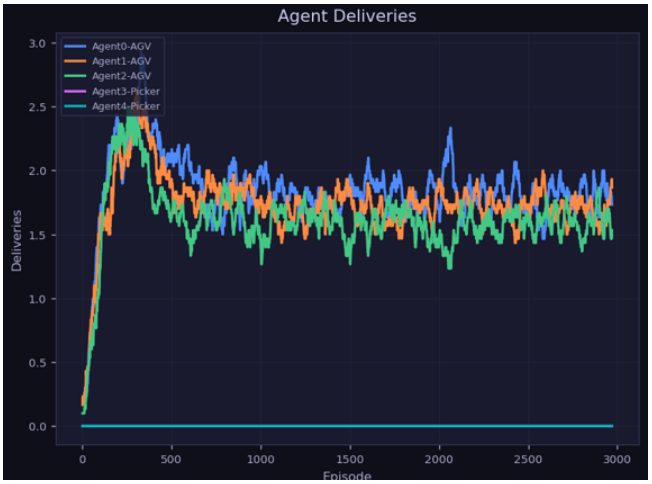


Fig. 7. Individual agent deliveries curve during the training phase

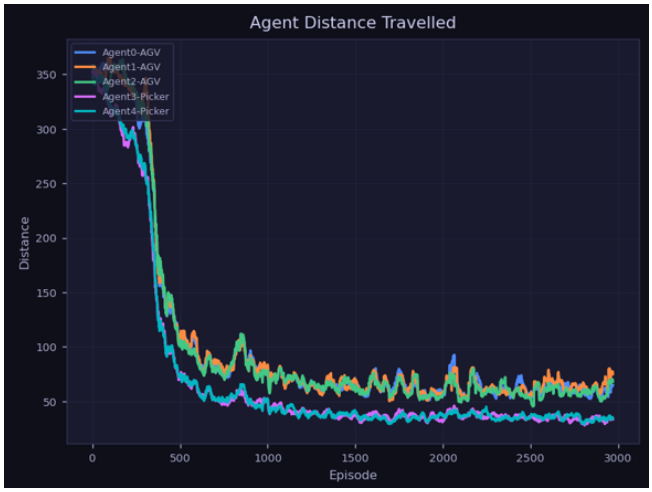


Fig. 8. Travelled distance in 3000 episodes

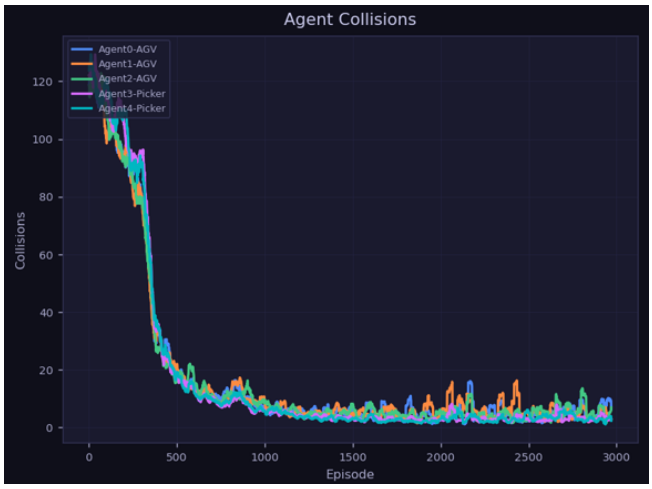


Fig. 9. Collisions during training

converged to zero showing that the agents learning not to cause any collisions as seen in Fig 9.

## IX. Results and Analysis

DDQN Policy Evaluation A policy evaluation for DDQN was carried out through an experiment using deterministic seeds where the DDQN policy was compared with two other baselines: the greedy and random policies. From the results obtained, the DDQN policy has proven to be better than both the greedy and random policies in terms of the average reward, success rate of deliveries, and effectiveness in managing the warehouse operations. In the case of DDQN, the average reward obtained is +154.6 with a success rate of 83.3%. The greedy algorithm gave a very poor result with a success rate of 3.3% while the random policy could not be successful at all. The actions of the agents were very well coordinated with low cases of failures.

However, the main challenge is the lack of coordination among the agents due to the choice of designing the learning agent independently.

Figure 11 illustrates the comparison between DDQN, greedy, and random policies.

In addition to the improvements in reward per episode, task completion, and Q value stabilization, training curves also revealed steady improvement in other parameters. The smoke run analysis indicated that the neural network was able to learn navigation tasks even in short training episodes. Apart from the mean rewards and success rates, six other performance measurements are provided by the evaluation process. The average number of deliveries carried out by the DQN policy is 5.0 deliveries per episode; its throughput is 6.17 deliveries per 100 steps, delivery cycle time is 64 steps, and visits to chargers amount to 0.67 per episode. All these statistics indicate the proper functioning of the battery management override feature. The greedy policy performs 0.33 deliveries per episode; its throughput is 0.056 deliveries per 100 steps, while the random policy does not deliver anything in any evaluation episodes. The DQN policy produces 100 collisions per episode, whereas the greedy one generates only

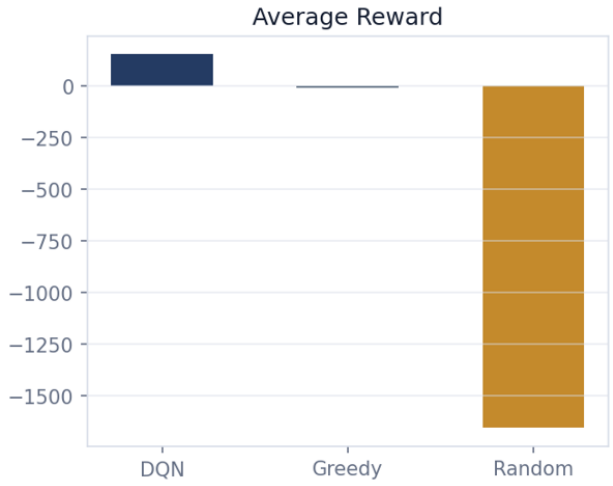


Fig. 10. Policy comparison showing DDQN outperforming greedy and random baselines in reward.

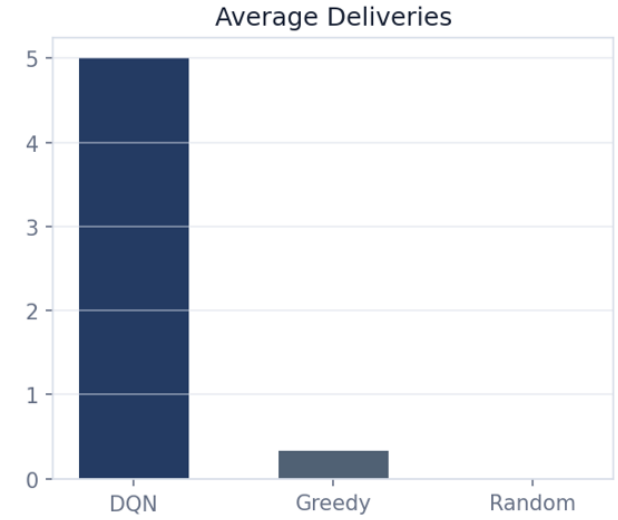


Fig. 11. Policy comparison showing DDQN outperforming greedy and random baselines in average delivery.

0.33 collisions. It happens because of the strategy used by the agents, who do not cooperate when pursuing their deliveries.

## X. Reflection, Limitations and Future Scope

There were a number of significant design considerations that contributed significantly to better performance in TA-RWARE. For instance, automatic pick up/drop transitions had a great impact on the improvement since it helped to eliminate sparse reward horizon and facilitated credit assignments. Shaping rewards also contributed towards quick navigation training thanks to dense guidance during exploration.

The battery emergency overrides did not allow learning charging behavior independently; as a result, the state-action search space was greatly reduced. Meanwhile, deterministic seeding and reproduction allowed consistent experiment evaluation, whereas Streamlit made it possible to visualize the results in real time.

However, there are some limitations to this framework as well. Specifically, independent learners do not have built-in coordination capabilities, which leads to relatively high collision rates at dense traffic in warehouses. Furthermore, the 5×5 observation range does not allow agents to see faraway chargers and order queues. Finally, the uniform sampling strategy is biased towards rare delivery occurrences.

Further improvement ideas might include QMIX or MAPPO algorithms for centralized multi-agent training, graph neural networks for global warehouse perception, Prioritized Experience Replay for rare event sampling, and multi-objective reinforcement learning.

## XI. Conclusion

In this study, an autonomous routing and task scheduling problem for the automated warehouse environment is solved by employing Multi-agent Reinforcement Learning approach. The design of the model includes battery powered routing, coordination among diverse agents, insertion of tasks dynamically and Duelling Double Deep Q-network Learning within the Automated Warehousing framework.

It can be observed from the analysis that the designed Duelling Deep Q-network was more successful when compared to the greedy and the random models based on the number of deliveries made, time taken and rewards earned by the respective models. Some of the research gaps in Automated Warehouse Logistics through Deep Reinforcement Learning are discussed here.

## References

[1] V. Mnih et al., “Human-Level Control through Deep Reinforcement Learning,” Nature, 2015.
[2] Z. Wang et al., “Dueling Network Architectures for Deep Reinforcement Learning,” ICML, 2016.
[3] H. van Hasselt et al., “Deep Reinforcement Learning with Double Q-Learning,” AAAI, 2016.
[4] T. Schaul et al., “Prioritized Experience Replay,” ICLR, 2016.
[5] F. Christianos et al., “Shared Experience Actor-Critic for Multi-Agent RL,” NeurIPS, 2020.
[6] G. Papoudakis et al., “Benchmarking MARL Algorithms in Cooperative Tasks,” NeurIPS D&B, 2021.
[7] A. Krnjaic et al., “Scalable MARL for Warehouse Logistics with Robotic & Human Co-Workers,” arXiv, 2022.
[8] P. Gankin et al., “Multi-Agent DQN for AGV Routing in Modular Manufacturing,” Frontiers in Robotics and AI, 2022.
[9] H. Hu et al., “Toward Energy-Efficient Routing of Multiple AGVs with MARL,” MDPI, 2023.
[10] M. Löffler et al., “Picker Routing in AGV-Assisted Order Picking Systems,” INFORMS Journal on Computing, 2022.